

Helix Thready — Diagram Render Status

Table of contents

1. Result summary

2. Per-area breakdown

3. Tooling used (VERIFIED)

4. How to reproduce

5. Source change made during this render

6. Open items

Rev	Date	Author	Change
1	2026-07-22	swarm (diagrams)	Initial status: 104/104 <code>.mmd</code> sources rendered to sibling <code>.svg</code> via mermaid-cli

This file records the machine-verifiable state of the Mermaid diagram export step for the `docs/public/research/mvp/` tree. Per `[CONSTITUTION §11.4.65]` (md→HTML/PDF sync) the `.mmd` files are the **source of truth**; PNG/SVG export runs via the Docs Chain / `mermaid-cli` at build time. `scripts/render-diagrams.sh` is that export step and is safe to re-run.

Table of contents

1. Result summary
2. Per-area breakdown
3. Tooling used (VERIFIED)
4. How to reproduce
5. Source change made during this render
6. Open items

1. Result summary

`[RESEARCH]` **VERIFIED — every `.mmd` rendered.** All **104** Mermaid sources under `docs/public/research/mvp/**/diagrams/` have a sibling `.svg` that is a genuine mermaid-cli render (each contains the `aria-roledescription="..."` / `class="flowchart|..."` signature the renderer emits — none are hand-faked placeholders).

Metric	Count
<code>.mmd</code> sources under <code>**/diagrams/</code>	104
Sibling <code>.svg</code> that are real mermaid renders	104
Missing renders	0
Faked / placeholder SVGs	0
Deferred	0

Rendered PNG/SVG exported via Docs Chain ([CONSTITUTION §11.4.65]). SVG is the committed format (preferred — vector, diff-able). `.png` is available on demand via `scripts/render-diagrams.sh png`.

Note: the seven `.svg` under `design/assets/` (logos, launcher icons, footer slogan) are authored brand assets, **not** diagram renders — they have no `.mmd` source and are out of scope for this export step.

2. Per-area breakdown

Area (<code>**/diagrams/</code>)	<code>.mmd</code>	real <code>.svg</code>
<code>api/diagrams</code>	8	8
<code>architecture/diagrams</code>	18	18
<code>database/diagrams</code>	11	11
<code>database/materials/diagrams</code>	1	1
<code>deployment/diagrams</code>	14	14
<code>deployment/materials/diagrams</code>	1	1
<code>design/diagrams</code>	17	17
<code>development/diagrams</code>	11	11
<code>development/materials/diagrams</code>	1	1
<code>testing/diagrams</code>	9	9
<code>user-guides/diagrams</code>	13	13
Total	104	104

3. Tooling used (VERIFIED)

[RESEARCH] The render was performed on the host with locally-installed tooling — no network fetch was required:

Component	Version / path
<code>mmdc</code> (<code>@mermaid-js/mermaid-cli</code>)	11.16.0 (global; also on <code>PATH</code>)
Node.js	v24.18.0
Chromium (Puppeteer engine)	138.0.7204.168 at <code>/usr/bin/chromium</code> , driven <code>--no-sandbox --disable-gpu</code>

Because a system Chromium is present, `render-diagrams.sh` auto-detects it and writes a Puppeteer config pointing at it; if only the Puppeteer-bundled Chromium is available the script falls back to that. Where `mmdc` is absent the script falls back to `npm -y @mermaid-js/mermaid-cli`.

4. How to reproduce

From `docs/public/research/mvp/`:

```
scripts/render-diagrams.sh      # render ALL to .svg (preferred, idempotent)
scripts/render-diagrams.sh png  # render ALL to .png instead
scripts/render-diagrams.sh svg api # only the api/ area, as .svg
```

The script exits non-zero and lists any file that fails to parse/render, so it doubles as a pre-tag diagram gate. Single file, matching what the swarm ran:

```
mmdc -i <in.mmd> -o <out.svg> -p <puppeteer-config.json>
# puppeteer-config.json: {"executablePath":"/usr/bin/chromium","args":["--no-sandbox","--disable-gpu"]}
```

5. Source change made during this render

[RESEARCH] One source needed a **Mermaid-syntax fix** to parse — a genuine defect, not a tooling gap: `development/materials/diagrams/sdk-publish-flow.mmd` used unquoted `{` inside two square-bracket node labels (`GENCORE[gen/{go,rust,dart} core]`), which the parser reads as a diamond-node start (`Parse error ... got 'DIAMOND_START'`). The labels were quoted (`GENCORE["gen/{go,rust,dart} core"]`) — identical displayed text, no semantic change — after which it renders cleanly. This is the same quoting convention already applied to sibling sources (e.g. labels containing `\n` such as `PP["port_prefix\n..."]`).

6. Open items

- [OPEN: `mmd-md-sync`] The `.mmd` files are the export source of truth, but the same Mermaid is also embedded inline in the area `.md` docs (per `CONVENTIONS §4`). The one syntax fix in §5 was applied to the `.mmd` only (this task's write scope is `diagrams/` , `scripts/` , and this status file). The

corresponding inline ``mermaid block in the owning .md should be re-synced by the doc owner so on-page rendering matches the exported SVG. [CONSTITUTION §11.4.65]`

- [OPEN: commit-artifacts] The 104 .svg renders and the quoting fixes to the .mmd sources are present in the working tree; committing/pushing them to all four upstreams is left to the operator per CONVENTIONS §8 ([CONSTITUTION §2.1]).

Made with love ♥ by Helix Development.